

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное
бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

**Лабораторная работа №3
Разработка веб-приложения на Spring Boot.
Введение в IoC/DI
по дисциплине
«Информационные технологии и программирование»**

Выполнил: студент гр. БВТ2403
Толибов Р.Ш.

Руководитель:

Москва, 2026 г.

Цель работы

Научиться создавать Spring Boot приложение, освоить принципы инверсии управления (IoC) и внедрения зависимостей (DI), изучить три способа конфигурации Springкомпонентов, а также научиться разрешать конфликты при наличии нескольких бинов одного типа

Задачи

- Создать проект Spring Boot с зависимостями Spring Web и Spring Boot DevTools.
- Разработать REST-контроллер с обработкой GET-запросов и параметров `@RequestParam`.
- Реализовать систему уведомлений без Spring (жёсткие связи) и выявить недостатки подхода.
- Перевести приложение на Spring с использованием явной Java-конфигурации (`@Configuration`, `@Bean`).
- Внедрить автоматическое сканирование бинов (`@Service`, `@ComponentScan`) и проанализировать проблему неоднозначности бинов.
- Освоить способы разрешения конфликтов: `@Primary`, `@Qualifier`, внедрение коллекции `List<MessageService>`.
- Выполнить самостоятельные задания для каждой части.

Ход работы

1. Знакомство с Spring Boot Web

Был создан проект через Spring Initializr с параметрами:

Project: Maven

Language: Java 17

Dependencies: Spring Web, Spring Boot DevTools

Название: spring-lab3-notifications

Структура пакетов:

org.example.controller, org.example.service, org.example.config.

В пакете controller создан класс HelloController:

```
package org.example.controller ;

import org.springframework.web.bind.annotation.GetMapping ;
import org.springframework.web.bind.annotation.RestController ;
import org.springframework.web.bind.annotation.RequestParam;

@RestController
public class HelloController {

    @GetMapping ("/hello")
    public String sayHello () {
        return "Привет, Spring Boot!";
    }

    @GetMapping ("/goodbye")
    public String sayGoodbye () {
        return "До свидания, Spring Boot!";
    }

    @GetMapping("/greet")
    public String greet(@RequestParam(value = "name", defaultValue = "Мир")
String name) {
        return String.format("Привет, %s!", name);
    }

    @GetMapping("/greet2")
    public String greet2(
        @RequestParam(value = "name", defaultValue = "Мир") String name,
        @RequestParam(value = "age", defaultValue = "0") int age) {
        return String.format("Пользователь: %s, Возраст: %d лет", name, age);
    }
}
```

Запуск главного класса Application (с аннотацией @SpringBootApplication) позволил проверить в браузере эндпоинты:

http://localhost:8080/hello → Привет, Spring Boot!

http://localhost:8080/goodbye → До свидания, Spring Boot!

http://localhost:8080/greet?name=Иван → Привет, Иван!

http://localhost:8080/userinfo?name=Анна&age=25 → Пользователь: Анна, возраст: 25

2. Проблема жёстких связей (без Spring)

Создан интерфейс `MessageService` и две его реализации:

```
public interface MessageService {
    void sendMessage(String message, String recipient);
}

public class EmailService implements MessageService {
    @Override
    public void sendMessage(String message, String recipient) {
        System.out.println("EMAIL to " + recipient + ": " + message);
    }
}

public class SmsService implements MessageService {
    @Override
    public void sendMessage(String message, String recipient) {
        System.out.println("SMS to " + recipient + ": " + message);
    }
}
```

Класс `NotificationManager` с жёсткой привязкой к `EmailService`:

```
public class NotificationManager {
    private final EmailService emailService;
    public NotificationManager() {
        this.emailService = new EmailService();
    }
    public void notify(String message, String recipient) {
        emailService.sendMessage(message, recipient);
    }
}
```

Контроллер `NotificationController` создавал экземпляр `NotificationManager` через `new`:

```
@RestController
public class NotificationController {
    @GetMapping("/notify")
    public String notify(@RequestParam String message, @RequestParam String email) {
        NotificationManager manager = new NotificationManager();
        manager.notify(message, email);
        return "Уведомление отправлено (жесткая связь)";
    }
}
```

Результат: при обращении на `http://localhost:8080/notify?message=Hi&email=test@test.com` в консоль выводилось EMAIL to test@test.com: Hi.

Модификация NotificationManager для работы с любым MessageService

Изменён конструктор и поле:

```
public class NotificationManager {
    private final MessageService messageService;
    public NotificationManager(MessageService messageService) {
        this.messageService = messageService;
    }
    public void notify(String message, String recipient) {
        messageService.sendMessage(message, recipient);
    }
}
```

Добавлен третий сервис:

```
public class PushService implements MessageService {
    @Override
    public void sendMessage(String message, String recipient) {
        System.out.println("PUSH to " + recipient + ": " + message);
    }
}
```

3. Java Config (явная конфигурация через @Configuration и @Bean)

```
package org.example.config;

import org.example.service.EmailService;
import org.example.service.NotificationManager;
import org.example.service.SmsService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean
    public EmailService emailService() {
        return new EmailService();
    }

    @Bean
    public SmsService smsService() {
        return new SmsService();
    }

    @Bean
    public NotificationManager notificationManager() {
```

```

        return new NotificationManager(emailService());
    }
}

```

NotificationManager изменён так, чтобы принимать MessageService через конструктор (как в самостоятельном задании части 2).

Контроллер теперь получает NotificationManager из контекста Spring:

```

@RestController
public class NotificationController {
    private final ApplicationContext context;

    public NotificationController(ApplicationContext context) {
        this.context = context;
    }

    @GetMapping("/notify")
    public String notify(@RequestParam String message, @RequestParam String email) {
        NotificationManager manager =
context.getBean(NotificationManager.class);
        manager.notify(message, email);
        return "Уведомление отправлено через Java Config";
    }
}

```

Главный класс импортирует AppConfig:

```

@SpringBootApplication
@Import(AppConfig.class)
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}

```

В методе notificationManager() заменён вызов smsService() вместо emailService(). Уведомления стали приходить через SMS.

Добавление PushService в конфигурацию:

```

@Bean
public PushService pushService() {
    return new PushService();
}

@Bean
public NotificationManager notificationManager() {
    return new NotificationManager(pushService());
}

```

Дополнительное задание: создание AnotherConfig:

```

@Configuration
public class AnotherConfig {

```

```

@Bean
public SmsService smsService() {
    return new SmsService();
}
}

```

Главный класс: `@Import({AppConfig.class, AnotherConfig.class})`. В `AppConfig` убран `smsService()`. Приложение продолжило работать – Spring объединяет бины из всех импортированных конфигураций.

4. Конфигурация через аннотации (`@Service`) и проблема конфликта

Сервисы отмечены `@Service`. `NotificationManager` также получил `@Service` и внедрение через конструктор (аннотация `@Autowired` не обязательна при единственном конструкторе).

```

@Service
public class NotificationManager {
    private final MessageService messageService;
    public NotificationManager(MessageService messageService) {
        this.messageService = messageService;
    }
    // ...
}

```

Контроллер упрощён – теперь можно внедрять `NotificationManager` напрямую:

```

@RestController
public class NotificationController {
    private final NotificationManager notificationManager;

    public NotificationController(NotificationManager notificationManager) {
        this.notificationManager = notificationManager;
    }

    @GetMapping("/notify")
    public String notify(@RequestParam String message, @RequestParam String email) {
        notificationManager.notify(message, email);
        return "Уведомление отправлено (аннотации)";
    }
}

```

В `AppConfig` удалены все `@Bean` методы, оставлен только `@ComponentScan("org.example")`. Главный класс по-прежнему импортирует `AppConfig`.

Результат: при запуске возникает ошибка: No qualifying bean of type 'MessageService' available: expected single matching bean but found 2: emailService, smsService

5. Разрешение конфликта бинов

Способ 1. @Primary

Добавлена аннотация @Primary над EmailService:

```
@Service
@Primary
public class EmailService implements MessageService { ... }
```

Ошибка исчезла, и по умолчанию стал использоваться EmailService.

Способ 2. @Qualifier

Убран @Primary. В конструкторе NotificationManager указан @Qualifier:

```
@Service
public class NotificationManager {
    private final MessageService messageService;

    public NotificationManager(@Qualifier("smsService") MessageService
messageService) {
        this.messageService = messageService;
    }
}
```

Теперь явно выбран SmsService.

Способ 3. Внедрение коллекции

Изменён NotificationManager для работы со всеми сервисами:

```
@Service
public class NotificationManager {
    private final List<MessageService> messageServices;

    public NotificationManager(List<MessageService> messageServices) {
        this.messageServices = messageServices;
    }

    public void notify(String message, String recipient) {
        messageServices.forEach(service -> service.sendMessage(message,
recipient));
    }
}
```

При запуске уведомление отправляется и по email, и по SMS одновременно.

Вывод

В ходе лабораторной работы было создано веб-приложение на Spring Boot, демонстрирующее принципы IoC и DI. Последовательно рассмотрены три способа конфигурации Spring-компонентов: явное объявление бинов в Java-конфигурации (@Configuration, @Bean), автоматическое сканирование с аннотациями (@Service, @ComponentScan), а также комбинированный подход.

Была выявлена и проанализирована проблема жёстких связей на примере системы уведомлений. Рефакторинг с использованием Spring показал преимущества внедрения зависимостей: гибкость, упрощение тестирования, централизованное управление объектами.

Освоены способы разрешения конфликтов при наличии нескольких бинов одного типа: @Primary, @Qualifier и внедрение коллекции. Наиболее универсальным признано внедрение Map<String, MessageService>, позволяющее динамически выбирать реализацию.

<https://github.com/TRuslan666/ITaP/tree/main/лаб3>